

Phase 1 - Week 1 - Lesson Day 2

NumPy Basics

Today's Objectives

Yesterday you refreshed core Python. Today you meet the tool that almost everything else in data science is built on top of: **NumPy**. Pandas, Scikit-learn, and even deep learning frameworks all use NumPy arrays under the hood, so getting comfortable with them now saves you confusion later.

- Understand what NumPy is and why it's faster than plain Python lists
- Create 1D, 2D, and 3D arrays
- Index and slice arrays to pull out specific values
- Understand array shape and how to reshape an array

1. What Is NumPy, and Why Use It?

Definition: NumPy (Numerical Python) is a library for working with arrays of numbers very fast. A plain Python list is flexible - it can hold mixed types, and grows/shrinks freely - but that flexibility costs speed, because Python has to store extra information about every single item. A NumPy array stores only raw numbers of a single fixed type, packed tightly together in memory. This is what makes NumPy dramatically faster for the kind of heavy numeric work data science needs.

```
import numpy as np    # 'np' is the standard alias used everywhere
```

Where it matters in data science: every dataset you load, every model you train, and every image or numeric feature you process is eventually represented as a NumPy array (or a Pandas DataFrame, which is itself built on top of NumPy arrays column by column).

2. Creating Arrays

You can build an array from a Python list, or use NumPy's built-in shortcuts to generate common arrays without typing out every value.

From a list:

```
arr1 = np.array([1, 2, 3, 4])           # 1D array - a single row of numbers

arr2 = np.array([[1, 2, 3], [4, 5, 6]]) # 2D array - a list of lists, like a table/matrix

arr3 = np.array([[[1, 2], [3, 4]],
                  [[5, 6], [7, 8]]])    # 3D array - a list of 2D arrays
```

1D means a single line of numbers (like a list). **2D** means rows and columns (like a spreadsheet or a grayscale image). **3D** adds a third dimension (like a stack of 2D grids, or a color image with red/green/blue

layers).

Built-in shortcuts:

```
zeros = np.zeros((2, 3))      # a 2x3 array filled entirely with 0
ones = np.ones((3, 3))       # a 3x3 array filled entirely with 1
range_arr = np.arange(0, 10, 2) # [0, 2, 4, 6, 8] - like range(), but returns an array
lin_arr = np.linspace(0, 1, 5) # 5 evenly spaced numbers between 0 and 1 (inclusive)
```

arange(start, stop, step) works like Python's `range()`, but gives you back a NumPy array instead of a plain iterator. **linspace(start, stop, count)** is different: instead of a step size, you tell it how many evenly-spaced numbers you want in total.

3. Indexing and Slicing

Definition: *Indexing* means pulling out a single value using its position. *Slicing* means pulling out a range of values using start:stop notation. This works similarly to Python lists, but NumPy extends it to work across multiple dimensions at once.

1D array indexing and slicing:

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])      # 10  -> first element
print(arr[-1])     # 50  -> last element
print(arr[1:4])    # [20, 30, 40] -> from index 1 up to (not including) index 4
```

2D array indexing (rows and columns): you provide two positions, separated by a comma: **array[row, column]**.

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print(matrix[0, 1]) # 2 -> row 0, column 1
print(matrix[:, 0]) # [1, 4, 7] -> the ':' means 'all rows', column 0 only
print(matrix[1, :]) # [4, 5, 6] -> row 1, the ':' means 'all columns'
```

Think of the colon : as meaning "everything along this dimension." This pattern of selecting whole rows or whole columns is something you will use constantly once you start working with real tables of data in Pandas.

4. Shape and Reshaping

Definition: **.shape** is a property of every array that tells you its dimensions, as a tuple - for example (3, 4) means 3 rows and 4 columns. **.reshape()** lets you change an array's dimensions without changing its actual data or order - as long as the total number of elements stays exactly the same before and after.

```
arr = np.arange(12)      # [0, 1, 2, ..., 11] -> a 1D array of 12 numbers
print(arr.shape)         # (12,) -> one dimension, 12 elements

matrix = arr.reshape(3, 4) # reshape into 3 rows, 4 columns (3 x 4 = 12, so it fits)
print(matrix.shape)       # (3, 4)
print(matrix)
```

If the numbers don't match - for example trying to reshape 12 elements into a (3, 5) shape (which needs 15 elements) - NumPy will raise an error. Always check that rows × columns equals the original total count before reshaping.

Today's Exercise

Open the accompanying notebook file **day-2-numpy-basics.ipynb** and complete the exercises below inside it. Read what each one is asking, then write the code yourself before checking back here if you get stuck.

Part A - Creating Arrays (3 questions)

1. Build a 2D array

Create a 2D array (matrix) representing a 3x3 grid with the numbers 1 through 9 filled in row by row, then print it.

2. Zeros and ones

Create a 4x2 array of all zeros, and a 2x4 array of all ones. Print both and compare their shapes.

3. arange and linspace

Create an array of even numbers from 0 to 20 using `arange()`, and a separate array of 10 evenly spaced numbers between 0 and 100 using `linspace()`.

Part B - Indexing and Slicing (4 questions)

4. Slice a 1D array

Given an array of numbers 0 to 19 (use `arange`), slice out only the numbers from index 5 to index 14.

5. Row and column access

Using the 3x3 matrix you built in Question 1, print the entire second row, then print the entire third column.

6. Single value lookup

Using the same 3x3 matrix, print the value in the middle of the grid (row 1, column 1) using indexing.

7. Reverse an array

Given an array of numbers 1 to 10, print it in reverse order. Hint: think about slicing with a negative step, e.g. `array[::-1]`.

Part C - Shape and Reshaping (3 questions)

8. Check the shape

Create an array using `arange(1, 16)` (numbers 1 to 15) and print its shape before reshaping anything.

9. Reshape it

Reshape that same array of 15 numbers into a 3x5 matrix, and separately into a 5x3 matrix. Print both and compare how the numbers are arranged differently.

10. Reshape error

Try reshaping the 15-number array into a (4, 4) shape (16 elements) and observe the error NumPy gives you. Write one sentence in a comment explaining why it failed.

How to Approach It

For indexing and slicing questions, always ask yourself: "am I working with rows, columns, or both?" and write out the shape you expect on paper first if it helps. For reshaping questions, always multiply the target dimensions together and check it matches your original element count before running the code - this habit will save you from a lot of avoidable errors later on.

Once you and Halimo have both completed the exercises in the notebook, bring your answers back to the lesson chat and we'll review them together before moving on to Lesson Day 3 (NumPy Operations).